# Flask

**Status:** not covered by Software Lab-1 practicals **Where it is used:** `backend/app.py` only **Closest lab concept:** Practical 4, functions and files — a Flask route is a function that takes a request and returns a response

---

## 1. What it is

Flask is a Python web framework. It turns ordinary functions into HTTP endpoints: a URL plus a function that handles requests to it.

```
from flask import Flask

app = Flask(__name__)

@app.get("/api/health")
def health():
    return {"status": "ok"}
```

Running it:

```
if __name__ == "__main__":
    app.run(host="127.0.0.1", port=8000, debug=True)
```

`127.0.0.1:8000` means this computer only, port 8000. Nothing is exposed to a network.

---

## 2. Why the project needed it

The brief separates the frontend from the backend. The frontend is a browser page; the backend is Python. A browser cannot call a Python function directly, so something has to sit between them and translate HTTP requests into function calls. That is Flask's whole job.

Without it, the only options would be a command line that the frontend cannot talk to, or writing an HTTP server from scratch with `socket`.

---

## 3. The four pieces actually used

### 3.1 The decorator

```
@app.get("/api/patients/<patient_id>")
def get_patient(patient_id):
    ...
```

`@app.get(...)` registers the function. The `<patient_id>` in the URL is a variable, captured by Flask and passed into the function as an argument.

So `GET /api/patients/P-0003` calls `get_patient("P-0003")`.

### 3.2 Reading the request body

```
body = request.get_json(silent=True) or {}
```

Returns the JSON body of a `POST` as a Python dict. `silent=True` returns `None` instead of raising when the body is missing or malformed, which is why the `or {}` is needed. This is defensive coding: an API receives requests from anywhere, including from software that sends garbage.

### 3.3 Reading query parameters

```
query = request.args.get("q", "")
```

Reads `?q=metformin` from the URL. The `""` is the default when the parameter is absent.

### 3.4 Returning JSON

```
return jsonify({"count": len(records)}), 200
return jsonify({"created": cleaned}), 201
return jsonify({"error": "..."}), 404
```

`jsonify` converts a Python dict into a JSON response and sets the correct `Content-Type` header. The second element is the HTTP status code, and returning it as a tuple is how a different code goes with the same body.

---

## 4. Status codes used, and why they differ

| Code | Meaning | Where in this project |
|---|---|---|
| 200 | Worked | most GET routes |
| 201 | Created | every POST that writes a row |
| 400 | Malformed request | blank search query, failed validation, no symptoms |
| 404 | Not found | unknown patient id, unknown dataset, unknown route |

Two distinctions worth defending, because both were mistakes caught during development:

**Blank query is 400, no matches is 200.** A search for `"zzzznothing"` returning zero results is an honest answer, so it is a success. A search with a blank `?q=` is a malformed request, so it is a client error.

**Contraindications are not errors.** A triage result that warns "Metformin is contraindicated for kidney disease" returns 200 with a `warnings` list. The request succeeded; the warning is part of the answer.

---

### 5. `app.test_client()`

Flask can call routes without starting a server:

```
client = app.test_client()

r = client.get("/api/health")
r.status_code          # 200
r.get_json()           # the parsed body
client.post("/api/patients", json={"name": "A"})
```

This is how `tests/test_app.py` exercises all 18 routes without opening a port. Much faster and more reliable than testing against a live server.

---

## 6. What Flask does *not* do here

`backend/app.py` contains no business logic. Every route calls one function from `store`, `registry`, `triage` or `analytics` and returns its result. Two reasons:

1. **Testability.** `triage.py` and `analytics.py` have full test suites that never import Flask. If scoring logic lived in routes, none of those tests could exist.
2. **Explainability.** For a lab submission, being able to say "the matching logic is in `triage.py`, and it is pure Python" is worth more than a few saved lines.

The one piece of logic in `app.py` is `full_record()`, which fills absent fields with blanks so a create response contains the whole schema. That belongs there because it is about the shape of the HTTP

response.

---

## 7. Debug mode

```
app.run(debug=True)
```

Gives detailed error pages and auto-reloads on save. Convenient in development, **must not** be used in production: it exposes code internals to anyone who can reach the port.

---

## 8. Questions this should answer

- What does the `@app.get` decorator do?
- Why does `jsonify` need a dict rather than a list?
- What is the difference between a 400 and a 404?
- Why does triage return 200 when it finds a contraindication?
- Why is there no scoring logic in `app.py`?